

Unit 8: Managing Resources and Scheduling Tasks

Overview of the Database Resource Manager

- The main goal of the Database Resource Manager is to give the Oracle database server more control over resource management decisions, thus circumventing problems resulting from inefficient operating system management.
- Oracle Database Resource Manager (the Resource Manager) enables you to manage multiple workloads within a database that are contending for system and database resources.
- Resource Manager allows the database to have more control over how hardware resources are allocated.
- When database resource allocation decisions are left to the operating system, you may encounter the following problems with workload management:

- *Excessive overhead*

Excessive overhead results from operating system context switching between Oracle Database server processes when the number of server processes is high.

- *Inefficient scheduling*

The operating system deschedules database servers while they hold latches, which is inefficient.

- *Inappropriate allocation of resources*

The operating system distributes resources equally among all active processes and cannot prioritize one task over another.

- *Inability to manage database-specific resources, such as parallel execution servers and active sessions*

How Does the Database Resource Manager Address These Problems?

The Resource Manager helps to overcome these problems by allowing the database more control over how hardware resources are allocated. In an

environment with multiple concurrent user sessions that run jobs with differing priorities, all sessions should not be treated equally. The Resource Manager enables you to classify sessions into groups based on session attributes, and to then allocate resources to those groups in a way that optimizes hardware utilization for your application environment.

With the Resource Manager, you can:

- Guarantee certain sessions a minimum amount of CPU regardless of the load on the system and the number of users.
- Distribute available CPU by allocating percentages of CPU time to different users and applications. In a data warehouse, a higher percentage can be given to ROLAP (relational online analytical processing) applications than to batch jobs.
- Limit the degree of parallelism of any operation performed by members of a group of users.
- Manage the order of parallel statements in the parallel statement queue. Parallel statements from a critical application can be enqueued ahead of parallel statements from a low priority group of users.
- Limit the number of parallel execution servers that a group of users can use. This ensures that all the available parallel execution servers are not allocated to only one group of users.
- Create an active session pool. An *active session pool* consists of a specified maximum number of user sessions allowed to be concurrently active within a group of users. Additional sessions beyond the maximum are queued for execution, but you can specify a timeout period, after which queued jobs will terminate. The active session pool limits the total number of sessions actively competing for resources, thereby enabling active sessions to make faster progress.

- Monitor resources

Automatically record statistics about resource usage. You can examine these statistics using real-time SQL monitoring and the Resource Manager dynamic performance views (the `V$RSRC_*` views).

- Limit the amount of PGA memory used by each session that belongs to a group of users.
- Manage runaway sessions or calls in the following ways:

- By detecting when a session or call consumes more than a specified amount of CPU, physical I/O, logical I/O, or elapsed time, and then automatically either terminating the session or call, or switching to a consumer group with a lower resource allocation or a limit on the percentage of CPU that the group can use. The SQL statements that are terminated due to their excessive consumption of system resources are *quarantined*, that is, they are not allowed to run again by generating compilation errors during their subsequent runs.

A logical I/O, also known as a buffer I/O, refers to reads and writes of buffers in the buffer cache. When a requested buffer is not found in memory, the database performs a physical I/O to copy the buffer from either disk or the flash cache into memory, and then a logical I/O to read the cached buffer.

- By recording detailed information about SQL statements that consume more than a specified amount of CPU, physical I/O, logical I/O, or elapsed time with real-time SQL monitoring.
- By using the Automatic Workload Repository (AWR) to analyze a persistent record of SQL statements that consume more than a specified amount of CPU, physical I/O, logical I/O, or elapsed time.

- By logging information about a runaway session without taking any other action related to the session.
- Prevent the execution of operations that the optimizer estimates will run for a longer time than a specified limit.
- Limit the amount of time that a session can be idle. This can be further defined to mean only sessions that are blocking other sessions.
- Allow a database to use different resource plans, based on changing workload requirements. You can dynamically change the resource plan, for example, from a daytime resource plan to a nighttime resource plan, without having to shut down and restart the instance. You can also schedule a resource plan change with Oracle Scheduler.

The Elements of Resource Manager

- Resource Manager includes several elements that you can manage.
- The elements of the Resource Manager include resource consumer groups, resource plans, and resource plan directives.

Element	Description
Resource consumer group	A group of sessions that are grouped together based on resource requirements. The Resource Manager allocates resources to resource consumer groups, not to individual sessions.
Resource plan	A container for directives that specify how resources are allocated to resource consumer groups. You specify how the database allocates resources by activating a specific resource plan.
Resource plan directive	Associates a resource consumer group with a particular plan and specifies how resources are to be allocated to that resource consumer group.

You use the DBMS_RESOURCE_MANAGER PL/SQL package to create and maintain these elements. The elements are stored in tables in the data dictionary. You can view information about them with data dictionary views.

Using the Resource Manager

- You must have the system privilege ADMINISTER_RESOURCE_MANAGER to administer the Resource Manager. This privilege (with the ADMIN option) is granted to database administrators through the DBA role.
- Being an administrator for the Database Resource Manager allows you to execute all of the procedures in the DBMS_RESOURCE_MANAGER package. These are listed in the following table

Subprogram	Description
BEGIN SQL BLOCK Procedure	Indicates the start of a block of SQL statements to be treated as a group by resource manager
CALIBRATE IO Procedure	Calibrates the I/O capabilities of storage
CLEAR PENDING AREA Procedure	Clears the work area for the resource manager
CREATE CATEGORY Procedure	Creates a new resource consumer group category
CREATE CDB PLAN Procedure	Creates entries which define consolidation resource plans.
CREATE CDB PLAN DIRECTIVE Procedure	Creates the plan directives of the consolidation resource plan
CREATE CDB PROFILE DIRECTIVE Procedure	Creates the performance profile directives of the consolidation resource plan
CREATE CONSUMER GROUP Procedure	Creates entries which define resource consumer groups
CREATE PENDING AREA Procedure	Creates a work area for changes to resource manager objects
CREATE PLAN Procedure	Creates entries which define resource plans
CREATE PLAN DIRECTIVE Procedure	Creates resource plan directives

<u>CREATE SIMPLE PLAN Procedure</u>	Creates a single-level resource plan containing up to eight consumer groups in one step
<u>DELETE CATEGORY Procedure</u>	Deletes an existing resource consumer group category
<u>DELETE CDB PLAN Procedure</u>	Deletes the consolidation resource plan
<u>DELETE CDB PLAN DIRECTIVE Procedure</u>	Deletes the plan directive of the consolidation resource plan
<u>DELETE CDB PROFILE DIRECTIVE Procedure</u>	Deletes the performance profile directive of the consolidation resource plan
<u>DELETE CONSUMER GROUP Procedure</u>	Deletes entries which define resource consumer groups
<u>DELETE PLAN Procedure</u>	Deletes the specified plan as well as all the plan directives it refers to
<u>DELETE PLAN CASCADE Procedure</u>	Deletes the specified plan as well as all its descendants (plan directives, subplans, consumer groups)
<u>DELETE PLAN DIRECTIVE Procedure</u>	Deletes resource plan directives
<u>DEQUEUE PARALLEL STATEMENT Procedure</u>	Dequeues a parallel statement from the parallel statement queue
<u>END SQL BLOCK Procedure</u>	Indicates the end of a block of SQL statements that should be treated as a group by resource manager
<u>SET CONSUMER GROUP MAPPING Procedure</u>	Adds, deletes, or modifies entries for the login and run-time attribute mappings
<u>SET CONSUMER GROUP MAPPING PRI Procedure</u>	Creates the session attribute mapping priority list
<u>SET INITIAL CONSUMER GROUP Procedure</u>	Assigns the initial resource consumer group for a user (Caution: Deprecated Subprogram)
<u>SUBMIT PENDING AREA Procedure</u>	Submits pending changes for the resource manager
<u>SWITCH CONSUMER GROUP FOR SESS Procedure</u>	Changes the resource consumer group of a specific session

<u>SWITCH CONSUMER GROUP FOR USER Procedure</u>	Changes the resource consumer group for all sessions with a given user name
<u>SWITCH PLAN Procedure</u>	Sets the current resource manager plan
<u>UPDATE CATEGORY Procedure</u>	Updates an existing resource consumer group category
<u>UPDATE CDB AUTOTASK DIRECTIVE Procedure</u>	Updates the plan directives with regard to automated maintenance tasks
<u>UPDATE CDB DEFAULT DIRECTIVE Procedure</u>	Updates the default values for a consolidation plan
<u>UPDATE CDB PLAN Procedure</u>	Updates the consolidation resource plan
<u>UPDATE CDB PLAN DIRECTIVE Procedure</u>	Updates the plan directives for a consolidation resource plan
<u>UPDATE CDB PROFILE DIRECTIVE Procedure</u>	Updates the performance profile directives of the consolidation resource plan
<u>UPDATE CONSUMER GROUP Procedure</u>	Updates entries which define resource consumer groups
<u>UPDATE PLAN Procedure</u>	Updates entries which define resource plans
<u>UPDATE PLAN DIRECTIVE Procedure</u>	Updates resource plan directives
<u>VALIDATE PENDING AREA Procedure</u>	Validates pending changes for the resource manager

Create Resource Plans

- **CREATE_PLAN** Procedure creates entries which define resource plans.

Syntax

```
DBMS_RESOURCE_MANAGER.CREATE_PLAN (
  plan                IN  VARCHAR2,
  comment              IN  VARCHAR2 DEFAULT NULL,
  active_sess_pool_mth IN  VARCHAR2 DEFAULT 'ACTIVE_SESS_POOL_ABSOLUTE',
  parallel_degree_limit_mth IN  VARCHAR2 DEFAULT
                                'PARALLEL_DEGREE_LIMIT_ABSOLUTE',
  queueing_mth        IN  VARCHAR2 DEFAULT 'FIFO_TIMEOUT',
  mgmt_mth            IN  VARCHAR2 DEFAULT 'EMPHASIS',
  sub_plan            IN  BOOLEAN DEFAULT FALSE,
  max_iops            IN  NUMBER DEFAULT NULL,
  max_mbps            IN  NUMBER DEFAULT NULL);
```

Here's a breakdown of the parameters for this procedure:

1. **plan** (IN VARCHAR2): The name of the resource plan being created.
2. **comment** (IN VARCHAR2 DEFAULT NULL): A comment or description of the resource plan. This parameter is optional.
3. **active_sess_pool_mth** (IN VARCHAR2 DEFAULT 'ACTIVE_SESS_POOL_ABSOLUTE'): Method used to limit the number of active sessions. The default value is `ACTIVE_SESS_POOL_ABSOLUTE`.
4. **parallel_degree_limit_mth** (IN VARCHAR2 DEFAULT 'PARALLEL_DEGREE_LIMIT_ABSOLUTE'): Method used to limit the degree of parallelism for queries. The default value is `PARALLEL_DEGREE_LIMIT_ABSOLUTE`.
5. **queueing_mth** (IN VARCHAR2 DEFAULT 'FIFO_TIMEOUT'): Method used to queue sessions when the active session pool is full. The default value is `FIFO_TIMEOUT` (First In, First Out with Timeout).
6. **mgmt_mth** (IN VARCHAR2 DEFAULT 'EMPHASIS'): Method used to manage sessions within the plan. The default value is `EMPHASIS`.
7. **sub_plan** (IN BOOLEAN DEFAULT FALSE): Indicates whether the plan is a sub-plan. The default value is `FALSE`.
8. **max_iops** (IN NUMBER DEFAULT NULL): Maximum number of I/O operations per second (IOPS) allowed for the plan. This parameter is optional.
9. **max_mbps** (IN NUMBER DEFAULT NULL): Maximum megabytes per second (MBPS) allowed for the plan. This parameter is optional.

Example

BEGIN

```
DBMS_RESOURCE_MANAGER.CREATE_PLAN(

    plan            => 'my_resource_plan',

    comment         => 'This is a custom resource plan for managing resources.',

    active_sess_pool_mth      => 'ACTIVE_SESS_POOL_ABSOLUTE',

    parallel_degree_limit_mth => 'PARALLEL_DEGREE_LIMIT_ABSOLUTE',

    queueing_mth          => 'FIFO_TIMEOUT',

    mgmt_mth              => 'EMPHASIS',

    sub_plan              => FALSE,

    max_iops              => 1000,

    max_mbps              => 100

);
```

END;

In this example, a resource plan named *my_resource_plan* is created with default methods for managing active session pool, parallel degree limits, queueing, and session management. Additionally, the plan limits the maximum IOPS to 1000 and the maximum MBPS to 100.

The Default Maintenance Resource Manager Plan

Oracle provides preconfigured resource plans for different maintenance windows, and you can view and modify these plans using Oracle Enterprise Manager or the DBMS_RESOURCE_MANAGER package. For instance, you might see plans like 'WEEKNIGHT_WINDOW' and 'WEEKEND_WINDOW', which Oracle uses to schedule maintenance tasks during off-peak hours.

To see the details of existing resource plans, including the Default Maintenance Resource Manager Plan, you can query the DBA_RSRC_PLANS view:

```
SELECT plan, comments
FROM dba_rsrc_plans;
```

This will list all resource plans, including the default ones provided by Oracle.

The default Oracle Database Resource Manager plan that manages system resources is often associated with the DEFAULT_CONSUMER_GROUP_PLAN. This system-wide plan is predefined by Oracle and is not created using the DBMS_RESOURCE_MANAGER.CREATE_PLAN procedure. Instead, it is part of Oracle's default resource management configuration.

The DEFAULT_CONSUMER_GROUP_PLAN is designed to allocate resources among different consumer groups to ensure fair resource distribution and to prevent any single user or session from monopolizing system resources.

DEFAULT_CONSUMER_GROUP_PLAN is the system-wide, pre-defined resource plan that Oracle Database uses by default. It cannot be directly created or dropped, but you can examine its directives to understand resource allocation strategies.

Consumer Groups categorize sessions based on their purpose or priority. The default plan typically includes following consumer groups:

- **SYS_GROUP:** This group encompasses system maintenance tasks like Automatic Database Diagnostic Manager (ADDM) and background jobs. It's often given higher resource allocation to ensure smooth system operation.
- **DEFAULT_CONSUMER_GROUP:** This is the default group for regular user sessions. It inherits resource allocation settings from the DEFAULT_CONSUMER_GROUP_PLAN.
- **LOW_GROUP:** This group is intended for low-priority sessions like batch jobs or reports. It might have lower CPU or I/O allocation compared to other groups.
- **OTHER_GROUPS:** This is a catch-all group for sessions that don't belong to any explicitly defined group. Their resource allocation depends on the plan directives and may be lower than defined groups.

The default resource plan configuration aims to balance resource allocation between these consumer groups to ensure that maintenance tasks, user queries, and other sessions can coexist without degrading overall system performance.

- To **activate** the DEFAULT_CONSUMER_GROUP_PLAN as the current resource plan, you can use the DBMS_RESOURCE_MANAGER.SWITCH_PLAN procedure:

```
BEGIN
    DBMS_RESOURCE_MANAGER.SWITCH_PLAN( 'DEFAULT_CONSUMER_GROUP_PLAN' );
END;
```

Create Resource Plan Directives

- You use the CREATE_PLAN_DIRECTIVE procedure to create resource plan directives. Each directive belongs to a plan or subplan and allocates resources to either a consumer group or subplan.
- The DBMS_RESOURCE_MANAGER.CREATE_PLAN_DIRECTIVE procedure in Oracle Database is used to create resource plan directives, which define how resources are allocated to consumer groups or subplans within a resource plan.

Syntax

```

DBMS_RESOURCE_MANAGER.CREATE_PLAN_DIRECTIVE (
    plan                                IN VARCHAR2,
    group_or_subplan                    IN VARCHAR2,
    comment                             IN VARCHAR2 DEFAULT NULL,
    active_sess_pool_p1                 IN NUMBER    DEFAULT NULL,
    queueing_p1                         IN NUMBER    DEFAULT NULL,
    parallel_degree_limit_p1            IN NUMBER    DEFAULT NULL,
    switch_group                         IN VARCHAR2 DEFAULT NULL,
    switch_time                         IN NUMBER    DEFAULT NULL,
    switch_estimate                     IN BOOLEAN   DEFAULT FALSE,
    max_est_exec_time                   IN NUMBER    DEFAULT NULL,
    undo_pool                           IN NUMBER    DEFAULT NULL,
    max_idle_time                       IN NUMBER    DEFAULT NULL,
    max_idle_blocker_time               IN NUMBER    DEFAULT NULL,
    mgmt_p1                             IN NUMBER    DEFAULT NULL,
    mgmt_p2                             IN NUMBER    DEFAULT NULL,
    mgmt_p3                             IN NUMBER    DEFAULT NULL,
    mgmt_p4                             IN NUMBER    DEFAULT NULL,
    mgmt_p5                             IN NUMBER    DEFAULT NULL,
    mgmt_p6                             IN NUMBER    DEFAULT NULL,
    mgmt_p7                             IN NUMBER    DEFAULT NULL,
    mgmt_p8                             IN NUMBER    DEFAULT NULL,
    switch_io_megabytes                 IN NUMBER    DEFAULT NULL,
    switch_io_reqs                      IN NUMBER    DEFAULT NULL,
    switch_for_call                     IN BOOLEAN   DEFAULT NULL,
    parallel_server_limit                IN NUMBER    DEFAULT NULL,
    utilization_limit                   IN NUMBER    DEFAULT NULL,
    switch_io_logical                   IN NUMBER    DEFAULT NULL,
    switch_elapsed_time                 IN NUMBER    DEFAULT NULL,
    shares                             IN NUMBER    DEFAULT NULL,
    parallel_stmt_critical               IN VARCHAR2 DEFAULT NULL,
    session_pga_limit                   IN NUMBER    DEFAULT NULL,
    pq_timeout_action                   IN NUMBER    DEFAULT NULL,
    parallel_queue_timeout               IN NUMBER    DEFAULT NULL,);

```

Parameters of `CREATE_PLAN_DIRECTIVE`

- **plan:** The name of the resource plan to which this directive belongs.
- **group_or_subplan:** The consumer group or subplan to which the directive applies.
- **comment:** A description of the directive.
- **active_sess_pool_p1:** Specifies the maximum number of active sessions for the consumer group or subplan.
- **queueing_p1:** Specifies the number of sessions that can be queued for the consumer group or subplan.
- **parallel_degree_limit_p1:** Limits the degree of parallelism for the consumer group or subplan.
- **switch_group:** The consumer group to switch to when certain criteria are met (e.g., exceeding a maximum execution time).
- **switch_time:** The maximum allowed execution time (in seconds) for a session before it is switched to the `switch_group`.
- **switch_estimate:** If `TRUE`, the switch is based on estimated execution time.
- **max_est_exec_time:** The maximum estimated execution time (in seconds) for a session.
- **undo_pool:** The amount of undo space (in KB) allocated to the consumer group or subplan.
- **max_idle_time:** The maximum idle time (in seconds) for a session before it is terminated.
- **max_idle_blocker_time:** The maximum time (in seconds) a session can block other sessions while being idle before it is terminated.
- **mgmt_p1 to mgmt_p8:** Management parameters for resource allocation.
- **switch_io_megabytes:** The maximum I/O megabytes a session can consume before it is switched to the `switch_group`.
- **switch_io_reqs:** The maximum I/O requests a session can make before it is switched to the `switch_group`.
- **switch_for_call:** If `TRUE`, the session is switched to the `switch_group` at the end of the call.
- **parallel_server_limit:** The limit on the number of parallel servers that can be used by the consumer group or subplan.

- **utilization_limit:** Specifies the maximum resource utilization limit for the consumer group or subplan.
- **switch_io_logical:** The maximum logical I/O operations a session can perform before it is switched to the `switch_group`.
- **switch_elapsed_time:** The maximum elapsed time (in seconds) for a session before it is switched to the `switch_group`.
- **shares:** Specifies the number of shares allocated to the consumer group or subplan.
- **parallel_stmt_critical:** Specifies whether parallel statements are critical (YES or NO).
- **session_pga_limit:** Specifies the maximum amount of PGA memory (in KB) that a session can use.
- **pq_timeout_action:** Action to be taken when a parallel query times out.
- **parallel_queue_timeout:** Timeout (in seconds) for queuing parallel execution requests.

Example

Here's an example of creating a resource plan directive within a custom resource plan:

BEGIN

```

DBMS_RESOURCE_MANAGER.CREATE_PLAN_DIRECTIVE(
    plan                    => 'MY_CUSTOM_PLAN',
    group_or_subplan       => 'MY_CONSUMER_GROUP1',
    comment                => 'Directive for first custom consumer group.',
    active_sess_pool_p1    => 10,  -- Limit to 10 active sessions
    queueing_p1            => 5,   -- Allow 5 sessions to queue
    parallel_degree_limit_p1 => 2,  -- Limit parallel degree to 2
    switch_group           => 'LOW_PRIORITY_GROUP', -- Switch to low priority group
                        on certain conditions
    switch_time             => 3600, -- 1 hour max execution time
    switch_estimate         => TRUE,
    max_est_exec_time       => 7200, -- 2 hours max estimated execution time
    undo_pool               => 1024, -- 1 GB of undo space

```

```

max_idle_time          => 300,  -- 5 minutes of idle time
max_idle_blocker_time => 600,  -- 10 minutes of idle blocker time
mgmt_p1                => 75,   -- 75% management priority
switch_io_megabytes    => 1000, -- 1 GB I/O limit
switch_io_reqs         => 10000, -- 10,000 I/O requests
switch_for_call        => TRUE,
parallel_server_limit  => 4,    -- Limit to 4 parallel servers
utilization_limit      => 90,   -- 90% resource utilization limit
switch_io_logical      => 50000, -- 50,000 logical I/O operations
switch_elapsed_time    => 1800, -- 30 minutes max elapsed time
shares                 => 20,   -- 20 shares allocated
parallel_stmt_critical => 'YES', -- Parallel statements are critical
session_pga_limit      => 512000, -- 500 MB of PGA memory
pq_timeout_action      => 1,    -- Action to take on parallel query timeout
parallel_queue_timeout => 300  -- 5 minutes of parallel queue timeout
);
END;
```

This directive allocates resources to the consumer group `MY_CONSUMER_GROUP1` within the plan `MY_CUSTOM_PLAN`. It specifies limits on active sessions, queueing, parallel degree, execution times, idle times, I/O, memory, and resource utilization. Additionally, it defines actions for switching the consumer group under certain conditions.

By configuring these directives, you can effectively manage and prioritize database resources according to the specific needs of your application and workload.

Allocate Resources for Resource Plans

- Allocating resources for resource plans in Oracle Database involves using the Database Resource Manager. The Resource Manager allows you to control the allocation of resources, such as CPU, I/O, and parallel execution resources, to different sessions or groups of sessions.

- Oracle Database Resource Manager allows you to allocate resources to different parts of your database using resource plans and directives.
 - A **resource plan** is a container that holds directives for allocating resources. You can create multiple plans for a database, but only one can be active at a time.
 - **Directives** within a plan specify how resources are distributed. Each directive targets a specific consumer group or another sub-plan.
 - Consumer groups categorize database sessions based on shared characteristics, like function or workload type.
 - Directives allocate resources to these consumer groups.

Here's a general step-by-step guide to allocating resources for resource plans:

Step 1: Create Resource Consumer Groups

- Resource consumer groups are used to group user sessions that have similar resource requirements.

To control resource consumption, you can assign user sessions to *resource consumer groups*. A resource consumer group defines a set of users who have similar resource usage requirements. A resource consumer group also specifies a resource allocation method for each controlled resource.

You can view resource consumer groups and their associated attributes in the data dictionary view `DBA_RSRC_CONSUMER_GROUPS`. Each entry contains the following information:

- resource consumer group name
- the CPU resource allocation method for the group
- comment
- status (pending or active)
- statement about whether or not the resource consumer group is mandatory (and cannot be deleted)

Each user has a default resource consumer group. By default, all sessions owned by a user belong to that user's default resource consumer group.

Users can have the privilege to switch to different consumer groups. You can use PL/SQL procedures to switch resource consumer groups for a particular session, assuming you have the appropriate privileges. You also

can dynamically change resource allocation methods for a resource consumer group while the database is running.

One consumer group named `DEFAULT_CONSUMER_GROUP` always exists in the data dictionary. All sessions that do not explicitly belong to any group belong to `DEFAULT_CONSUMER_GROUP`.

Example:

```
BEGIN

  DBMS_RESOURCE_MANAGER.CREATE_CONSUMER_GROUP(

    CONSUMER_GROUP => 'oltp_group',

    COMMENT => 'OLTP Transactions Group'

  );

  DBMS_RESOURCE_MANAGER.CREATE_CONSUMER_GROUP(

    CONSUMER_GROUP => 'batch_group',

    COMMENT => 'Batch Processing Group'

  );

END;

/
```

The above code snippet successfully creates two consumer groups in Oracle Database Resource Manager:

- `oltp_group`: This group likely represents Online Transaction Processing (OLTP) workloads in your database. OLTP typically involves frequent short transactions.
- `batch_group`: This group likely represents batch processing workloads, which are long-running jobs that process large amounts of data.

By creating separate consumer groups, you can define different resource allocation strategies for these distinct workload types.

Step 2: Create Resource Plan

A resource plan defines how resources are allocated among the consumer groups.

Example:

```
BEGIN

    DBMS_RESOURCE_MANAGER.CREATE_PLAN(

        PLAN => 'mixed_workload_plan',

        COMMENT => 'Resource Plan for Mixed Workload'

    );

END;

/
```

Step 3: Create Plan Directives

Plan directives specify the resource allocations for each consumer group within a resource plan.

Example:

```
BEGIN

    DBMS_RESOURCE_MANAGER.CREATE_PLAN_DIRECTIVE(

        PLAN => 'mixed_workload_plan',

        GROUP_OR_SUBPLAN => 'oltp_group',

        COMMENT => 'OLTP Group Directive',

        MGMT_P1 => 80 -- Allocate 80% of CPU resources to OLTP group

    );
```

```

DBMS_RESOURCE_MANAGER.CREATE_PLAN_DIRECTIVE(

    PLAN => 'mixed_workload_plan',

    GROUP_OR_SUBPLAN => 'batch_group',

    COMMENT => 'Batch Group Directive',

    MGMT_P1 => 20  -- Allocate 20% of CPU resources to Batch group

);

END;

/

```

The above code snippet creates directives within a resource plan named `mixed_workload_plan` to allocate CPU resources to the previously defined consumer groups.

With this code, we have created a resource plan named `mixed_workload_plan` that allocates 80% of CPU resources to the `oltp_group` and 20% to the `batch_group`. This ensures that OLTP workloads, which likely require more CPU for frequent transactions, receive a larger share of resources.

Step 4: Validate the Resource Plan

Before you can use the resource plan, you must validate it to ensure that it does not contain any errors.

```

BEGIN

    DBMS_RESOURCE_MANAGER.VALIDATE_PENDING_AREA;

END;

/

```

Step 5: Submit the Pending Area

Once validated, submit the pending area to make the changes take effect.

```
BEGIN

    DBMS_RESOURCE_MANAGER.SUBMIT_PENDING_AREA;

END;

/
```

Pending Area?

The pending area is a staging area where you can create a new resource plan, update an existing plan, or delete a plan without affecting currently running applications.

When you create a pending area, the database initializes it and then copies existing plans into the pending area so that they can be updated.

```
BEGIN

    DBMS_RESOURCE_MANAGER.CREATE_PENDING_AREA();

END;

/
```

After you make changes in the pending area, you validate the pending area and then submit it. Upon submission, all pending changes are applied to the data dictionary, and the pending area is cleared and deactivated.

Submitting the pending area does not activate any new plan that you create; it just stores new or updated plan information in the data dictionary. However, if you modify a plan that is currently active, the plan is reactivated with the new plan definition.

Step 6: Activate the Resource Plan

Activate the resource plan by setting it as the current resource plan.

Example:

```
ALTER SYSTEM SET RESOURCE_MANAGER_PLAN = 'mixed_workload_plan';
```

- DBMS_RESOURCE_MANAGER.SUBMIT_PENDING_AREA; instructs the Resource Manager to activate the plan configurations accumulated in the work area.
- Once submitted, these changes become active and affect resource allocation for database sessions.

Step 7: Assign Users to Resource Consumer Groups

You need to assign users to the appropriate resource consumer groups.

Example:

```
EXEC
  DBMS_RESOURCE_MANAGER_PRIVS.GRANT_SWITCH_CONSUMER_GROUP(

  GRANTEE_NAME => 'hr',

  CONSUMER_GROUP => 'oltp_group',

  GRANT_OPTION => FALSE

);

EXEC
  DBMS_RESOURCE_MANAGER_PRIVS.GRANT_SWITCH_CONSUMER_GROUP(

  GRANTEE_NAME => 'batch_user',

  CONSUMER_GROUP => 'batch_group',

  GRANT_OPTION => FALSE

);
```

Step 8: Monitor Resource Usage

You can monitor the resource usage and the effectiveness of the resource plan using various dynamic performance views, such as V\$RSRC_CONSUMER_GROUP and V\$RSRC_PLAN.

```
SELECT * FROM V$RSRC_CONSUMER_GROUP;

SELECT * FROM V$RSRC_PLAN;
```

Introduction to The Scheduler

- Oracle Database includes Oracle Scheduler, an enterprise job scheduler to help you simplify the scheduling of hundreds or even thousands of tasks. Oracle Scheduler (the Scheduler) is implemented by the procedures and functions in the DBMS_SCHEDULER PL/SQL package.
- The Scheduler enables you to control when and where various computing tasks take place in the enterprise environment. The Scheduler helps you effectively manage and plan these tasks. By ensuring that many routine computing tasks occur without manual intervention, you can lower operating costs, implement more reliable routines, minimize human error, and shorten the time windows needed.
- The Scheduler provides **sophisticated, flexible enterprise scheduling functionality, which you can use to:**
 - Run **database program units**
You can run program units, that is, PL/SQL anonymous blocks, PL/SQL stored procedures, and Java stored procedures on the local database or on one or more remote Oracle databases.
 - Run external executables, (executables that are external to the database)
You can run **external executables**, such as applications, shell scripts, and batch files, on the local system or on one or more remote systems. Remote systems do not require an Oracle Database installation; they require only a Scheduler agent. Scheduler agents are available for all platforms supported by Oracle Database and some additional platforms.
 - Schedule job execution using the following methods:
 - Time-based scheduling
You can schedule a job to run at a particular date and time, either once or on a repeating basis. You can define complex repeat intervals, such as "every Monday and Thursday at 3:00 a.m. except on public holidays" or "the last Wednesday of each business quarter."
 - Event-based scheduling
You can start jobs in response to system or business events. Your applications can detect events and then signal the Scheduler.

Depending on the type of signal sent, the Scheduler starts a specific job. Examples of event-based scheduling include starting jobs when a file arrives on a system, when inventory falls below predetermined levels, or when a transaction fails. Beginning with Oracle Database 11g Release 2 (11.2), a Scheduler object called a file watcher simplifies the task of configuring a job to start when a file arrives on a local or remote system.

- Dependency scheduling

You can set the Scheduler to run tasks based on the outcome of one or more previous tasks. You can define complex dependency chains that include branching and nested chains.

- Prioritize jobs based on business requirements.

The Scheduler provides control over resource allocation among competing jobs, thus aligning job processing with your business needs. This is accomplished in the following ways:

- Controlling Resources by Job Class

You can group jobs that share common characteristics and behavior into larger entities called job classes. You can prioritize among the classes by controlling the resources allocated to each class. Therefore, you can ensure that your critical jobs have priority and enough resources to complete. For example, for a critical project to load a data warehouse, you can combine all the data warehousing jobs into one class and give it priority over other jobs by allocating a high percentage of the available resources to it. You can also assign relative priorities to the jobs within a job class.

- Controlling Job Prioritization based on Schedules

You can change job priorities based on a schedule. Because your definition of a critical job can change over time, the Scheduler also enables you to change the priority among your jobs over that time frame. For example, extract, transfer, and load (ETL) jobs used to load a data warehouse may be critical during non-peak hours but not during peak hours. Additionally, jobs that must run during the close of a business quarter may need to take priority over the

ETL jobs. In these cases, you can change the priority among the job classes by changing the resources allocated to each class.

- Manage and monitor jobs

You can manage and monitor the multiple states that jobs go through, from creation to completion. The Scheduler logs activity so that you can easily track information such as the status of the job and the last run time of the job by querying views using Oracle Enterprise Manager Cloud Control or SQL. These views provide valuable information about jobs and their execution that can help you schedule and better manage your jobs. For example, a DBA can easily track all jobs that failed for a particular user.

When you create a multiple-destination job, a job that is defined at one database but that runs on multiple remote hosts, you can monitor the status of the job at each destination individually or the overall status of the parent job as a whole.

For advanced job monitoring, your applications can subscribe to job state change notifications that the Scheduler delivers in event queues. The Scheduler can also send e-mail notifications when a job changes state.

- Execute and manage jobs in a clustered environment

A cluster is a set of database instances that cooperates to perform the same task. Oracle Real Application Clusters (Oracle RAC) provides scalability and reliability without any change to your applications. The Scheduler fully supports execution of jobs in such a clustered environment. To balance the load on your system and for better performance, you can also specify the database service where you want a job to run

Jobs and Supporting Scheduler Objects

- We use jobs and other scheduler objects for task scheduling.
- To use the Scheduler, you create *Scheduler objects*. Schema objects define the what, when, and where for job scheduling. Scheduler objects enable a modular approach to managing tasks. One advantage of the modular approach is that objects can be reused when creating new tasks that are similar to existing tasks.

- The principal Scheduler object is the job. A **job** defines the action to perform, the schedule for the action, and the location or locations where the action takes place. Most other scheduler objects are created to support jobs.
- Because Scheduler objects belong to schemas, you can grant object privileges on them. Some Scheduler objects, including job classes, windows, and window groups, are always created in the SYS schema, even if the user is not SYS. All other objects are created in the user's own schema or in a designated schema.

Programs

- A program object (program) describes what is to be run by the Scheduler.
- A *program includes*:
 - *An action*: For example, the name of a stored procedure, the name of an executable found in the operating system file system (an "external executable"), or the text of a PL/SQL anonymous block.
 - A
type: STORED_PROCEDURE, PLSQL_BLOCK, SQL_SCRIPT, EXTERNAL_SCRIPT, BACKUP_SCRIPT, or EXECUTABLE, where EXECUTABLE indicates an external executable.
 - *Number of arguments*: The number of arguments that the stored procedure or external executable accepts.

A program is a separate entity from a job. A job runs at a certain time or because a certain event occurred, and invokes a certain program. You can create jobs that point to existing program objects, which means that different jobs can use the same program and run the program at different times and with different settings. With the right privileges, different users can use the same program without having to redefine it. Therefore, you can create program libraries, where users can select from a list of existing programs.

If a stored procedure or external executable referenced by the program accepts arguments, you define these arguments in a separate step after creating the program. You can optionally define a default value for each argument.

Schedules

- A schedule object (schedule) specifies when and how many times a job is run.
- Schedules can be shared by multiple jobs. For example, the end of a business quarter may be a common time frame for many jobs. Rather than defining an end-of-quarter schedule each time a new job is defined, job creators can point to a named schedule.
- There are two types of schedules:
 - **time schedules**
With time schedules, you can schedule jobs to run immediately or at a later time. Time schedules include a start date and time, optional end date and time, and optional repeat interval.
 - **event schedules**
With event schedules, you can specify that a job executes when a certain event occurs, such as inventory falling below a threshold or a file arriving on a system. For more information on events.

Jobs

- A job object (job) is a collection of metadata that describes a user-defined task. It defines what must be executed (the action), when (the one-time or recurring schedule or a triggering event), where (the destinations), and with what credentials. A job has an owner, which is the schema in which it is created.
- A job that runs a database program unit is known as a **database job**. A job that runs an external executable is known as an **external job**.
- Jobs that run database program units at one or more remote locations are called **remote database jobs**. Jobs that run external executables at one or more remote locations are called **remote external jobs**.
- You define where a job runs by specifying a one or more destinations. Destinations are also Scheduler objects and are described later in this section. If you do not specify a destination, it is assumed that the job runs on the local database.

Specifying a Job Action

- ☞ You can specify a job action by specifying the database program unit or external executable to be run or the name of an existing program object (program).
- ☞ You specify the job action in one of the following ways:
 - By specifying as a job attribute the database program unit or external executable to be run. This is known as specifying the job action inline.
 - By specifying as a job attribute the name of an existing program, that specifies the database program unit or external executable to be run. The job owner must have the EXECUTE privilege on the program or the EXECUTE ANY PROGRAM system privilege.

Specifying a Job Schedule

- ☞ You can specify a job schedule by setting attributes of the job object or the name of an existing schedule object (schedule).
- ☞ You specify the job schedule in one of the following ways:
 - By setting attributes of the job object to define start and end dates and a repeat interval, or to define an event that starts the job. This is known as specifying the schedule **inline**.
 - By specifying as a job attribute the name of an existing schedule, which defines start and end dates and a repeat interval, or defines an event.

Specifying a Job Destination

- ☞ You can specify a job destination in several different ways.
- ☞ You specify the job destinations in one of the following ways:
 - By specifying as a job attribute a single named destination object. In this case, the job runs on one remote location.
 - By specifying as a job attribute a named destination group, which is equivalent to a list of remote locations. In this case, the job runs on all remote locations.
 - By not specifying a destination attribute, in which case the job runs locally. The job runs either of the following:
 - A database program unit on the local database (the database on which the job is created)

- An external executable on the local host, depending on the job action type

Specifying a Job Credential

- ☞ You can specify a job credential by specifying a named credential object or by allowing the credential attribute of the job to remain NULL.
- ☞ You specify the job credentials in one of the following ways:
 - By specifying as a job attribute a named credential object, which contains a database user name and password (for database jobs).
 - The job runs as the user named in the credential.
 - By allowing the credential attribute of the job to remain NULL, in which case a local database job runs as the job owner. The job owner is the schema in which the job was created.

Note:

A local database job always runs as the user is who is the job owner and will ignore any named credential.

- ☞ After you create a job and enable it, the Scheduler automatically runs the job according to its schedule or when the specified event is detected. You can view the run status of job and its job log by querying data dictionary views. If a job runs on multiple destinations, you can query the status of the job at each destination.

Destinations

- A destination object (destination) defines a location for running a job.
- There are two types of destinations:
 - *External destination:* Specifies a remote host name and IP address for running a remote external job.
 - *Database destination:* Specifies a remote database instance for running a remote database job.
- Jobs that run external executables (external jobs) must specify external destinations, and jobs that run database program units (database jobs) must specify database destinations.

- If you specify a destination when you create a job, the job runs on that destination. If you do not specify a destination, the job runs locally, on the system on which it is created.
- You can also create a destination group, which consists of a list of destinations, and reference this destination group when creating a job. In this case, the job runs on all destinations in the group.

Note:

Destination groups can also include the keyword `LOCAL` as a group member, indicating that the job also runs on the local host or local database.

- No object privileges are required to use a destination created by another user.

File Watchers

- A file watcher object (file watcher) defines the location, name, and other properties of a file whose arrival on a system causes the Scheduler to start a job.
- You create a file watcher and then create any number of event-based jobs or event schedules that reference the file watcher. When the file watcher detects the arrival of the designated file, it raises a file arrival event. The job started by the file arrival event can retrieve the event message to learn about the newly arrived file.
- A file watcher can watch for a file on the local system (the same host computer running Oracle Database) or a remote system, provided that the remote system is running the Scheduler agent.
- To use file watchers, the database Java virtual machine (JVM) component must be installed.

Credentials

- **Credentials** are user name and password pairs stored in a dedicated database object.
- Scheduler jobs use credentials to authenticate themselves with a database instance or the operating system in order to run. You use credentials for:
 - *Remote database jobs:* The credential contains a database user name and password. The stored procedure or PL/SQL block specified in the remote database job runs as this database user.

- *External jobs (local or remote):* The credential contains a host operating system user name and password. The external executable of the job then runs with this user name and password.
- *File watchers:* The credential contains a host operating system user name and password. The job that processes the file arrival event uses this user name and password to access the arrived file.
- You can query the *_CREDENTIALS views to see a list of credentials in the database. Credential passwords are stored obfuscated, and are not displayed in these views.

Chains

- Chains are the means by which you can implement dependency scheduling, in which job starts depend on the outcomes of one or more previous jobs.
- A chain consists of multiple steps that are combined using dependency rules. The dependency rules define the conditions that can be used to start or stop a step or the chain itself. Conditions can include the success, failure, or completion- or exit-codes of previous steps. Logical expressions, such as AND/OR, can be used in the conditions. In a sense, a chain resembles a decision tree, with many possible paths for selecting which tasks run and when.
- In its simplest form, a chain consists of two or more Scheduler program objects (programs) that are linked together for a single, combined objective. An example of a chain might be "run program A followed by program B, and then run program C only if programs A and B complete successfully, otherwise wait an hour and then run program D."
- As an example, you might want to create a chain to combine the different programs necessary for a successful financial transaction, such as validating and approving a loan application, and then funding the loan.
- A Scheduler job can point to a chain instead of pointing to a single program object. The job then serves to start the chain. This job is referred to as the **chain job**. Multiple chain jobs can point to the same chain, and more than one of these jobs can run simultaneously, thereby creating multiple instances of the same chain, each at a different point of progress in the chain.

- Each position within a chain is referred to as a **step**. Typically, after an initial set of chain steps has started, the execution of successive steps depends on the completion of one or more previous steps. Each step can point to one of the following:
 - A program object (program)
The program can run a database program unit (such as a stored procedure or PL/SQL anonymous block) or an external executable.
 - Another chain (a nested chain)
Chains can be nested to any level.
 - An event schedule, inline event, or file watcher
After starting a step that points to an event schedule or that has an inline event specification, the step waits until the specified event is raised. Likewise, a step that references a file watcher inline or that points to an event schedule that references a file watcher waits until the file arrival event is raised. For a file arrival event or any other type of event, when the event occurs, the step completes, and steps that are dependent on the event step can run. A common example of an event in a chain is a user intervention, such as an approval or rejection.
- Multiple steps in the chain can invoke the same program or nested chain.
- For each step, you can specify either a database destination or an external destination on which the step should run. If a destination is not specified, the step runs on the originating (local) database or the local host. Each step in a chain can run on a different destination.

Job Classes

- Job classes enable you to assign the same attributes to member jobs, set resource allocation for member jobs, and group jobs for prioritization.
- You typically create job classes only when you are in the role of Scheduler administrator.
- Job classes provide a way to:
 - Assign the same set of attribute values to member jobs

Each job class specifies a set of attributes, such as logging level. When you assign a job to a job class, the job inherits those attributes. For example, you can specify the same policy for purging log entries for all payroll jobs.

— Set service affinity for member jobs

You can set the service attribute of a job class to a desired database service name. This determines the instances in a Real Application Clusters environment that run the member jobs, and optionally, the system resources that are assigned to member jobs.

— Set resource allocation for member jobs

Job classes provide the link between the Database Resource Manager and the Scheduler, because each job class can specify a resource consumer group as an attribute. Member jobs then belong to the specified consumer group and are assigned resources according to settings in the current resource plan.

Alternatively, you can leave the `resource_consumer_group` attribute NULL and set the service attribute of a job class to a desired database service name. That service can in turn be mapped to a resource consumer group. If both the `resource_consumer_group` and service attributes are set, and the designated service maps to a resource consumer group, the resource consumer group named in the `resource_consumer_group` attribute takes precedence.

— Group jobs for prioritization

Within the same job class, you can assign priority values of 1-5 to individual jobs so that if two jobs in the class are scheduled to start at the same time, the one with the higher priority takes precedence. This ensures that you do not have a less important job preventing the timely completion of a more important one.

If two jobs have the same assigned priority value, the job with the earlier start date takes precedence. If no priority is assigned to a job, its priority defaults to 3.

- When defining job classes, try to classify jobs by functionality. Consider dividing jobs into groups that access similar data, such as marketing, production, sales, finance, and human resources.
- Some of the restrictions to keep in mind are:
 - A job must be part of exactly one class. When you create a job, you can specify which class the job is part of. If you do not specify a class, the job automatically becomes part of the class `DEFAULT_JOB_CLASS`.
 - Dropping a class while there are still jobs in that class results in an error. You can force a class to be dropped even if there are still jobs that are members of that class, but all jobs referring to that class are then automatically disabled and assigned to the class `DEFAULT_JOB_CLASS`. Jobs belonging to the dropped class that are already running continue to run under class settings determined at the start of the job

Windows

- A window is an interval of time to run a job.
- You create windows to automatically start jobs or to change resource allocation among jobs during various time periods of the day, week, and so on. A window is represented by an interval of time with a well-defined beginning and end, such as "from 12am-6am".
- You typically create windows only when you are in the role of Scheduler administrator.
- Windows work with job classes to control resource allocation. Each window specifies the resource plan to activate when the window opens (becomes active), and each job class specifies a resource consumer group or specifies a database service, which can map to a consumer group. A job that runs within a window, therefore, has resources allocated to it according to the consumer group of its job class and the resource plan of the window.

Groups

- A group designates a list of Scheduler objects.
- Instead of passing a list of objects as an argument to a DBMS_SCHEDULER package procedure, you create a group that has those objects as its members, and then pass the group name to the procedure.
- There are three types of groups:
 - Database destination groups: Members are database destinations, for running remote database jobs.
 - External destination groups: Members are external destinations, for running remote external jobs.
 - Window groups: Members are Scheduler windows.
- All members of a group must be of the same type and each member must be unique.
- You create a group with the DBMS_SCHEDULER.CREATE_GROUP procedure.

Incompatibilities

- An incompatibility definition (or, incompatibility) specifies incompatible jobs or programs, where only one of the group can be running at a time.
- For example, if jobs A and B are defined as incompatible, the Scheduler ensures that only one of them can be running at any given time, even if their respective job schedules would otherwise cause them to run at the same time.
- An incompatibility can be defined at the job level (the default) or the program level. For example, assume the following:
 - Jobs J1 and J2 are based on program P1.
 - Jobs J3, J4, and J5 are based on program P2.
 - Jobs J6 and J7 are based on program P3.

In this scenario:

- If a job-level incompatibility definition specifies J3, J4, and J5, and if job J3 is running, then J4 and J5 cannot be running until J3 finishes.
- If a program-level incompatibility definition specifies P1, P2, and P3, jobs J1 and J2 can run simultaneously (unless a job-level constraint prevents J1

and J2 from running simultaneously); however, no jobs based on programs P2 and P3 can be running until all jobs based on P1 finish.

Access Rights

- In Oracle Database, managing access rights for the Oracle Scheduler involves granting specific privileges to users to enable them to create, manage, and execute jobs. Here's a detailed breakdown of how to handle access rights in Oracle Scheduler:
- Some Scheduler Privileges
 - **CREATE JOB:** Allows the user to create jobs in their schema.
 - **CREATE ANY JOB:** Allows the user to create jobs in any schema.
 - **MANAGE SCHEDULER:** Allows the user to manage Scheduler objects.
 - **EXECUTE ANY PROGRAM:** Allows the user to execute any program.
 - **EXECUTE ANY CLASS:** Allows the user to execute any job class.
 - **ALTER:** Allows altering of scheduler objects.
 - **DROP:** Allows dropping of scheduler objects.
 - **SELECT ANY DICTIONARY:** Allows the user to select from any dictionary view, which can be necessary for certain Scheduler queries.

Granting Scheduler Privileges

To grant these privileges to a user, you use the GRANT statement. Here are examples of how to grant the necessary privileges to a user:

```
-- Grant the ability to create jobs in their schema
```

```
GRANT CREATE JOB TO username;
```

```
-- Grant the ability to create jobs in any schema
```

```
GRANT CREATE ANY JOB TO username;
```

```
-- Grant the ability to manage scheduler objects
```

```
GRANT MANAGE SCHEDULER TO username;
```

```
-- Grant the ability to execute any program
```

```
GRANT EXECUTE ANY PROGRAM TO username;
```

```
-- Grant the ability to execute any job class
```

```
GRANT EXECUTE ANY CLASS TO username;
```

Scheduler Components and Workflow

- Oracle Scheduler in Oracle provides a robust framework for automating tasks within your database. It achieves this through a combination of interrelated components and a well-defined workflow

Scheduler Components

Scheduler components include the job table, the job coordinator, and job slaves.

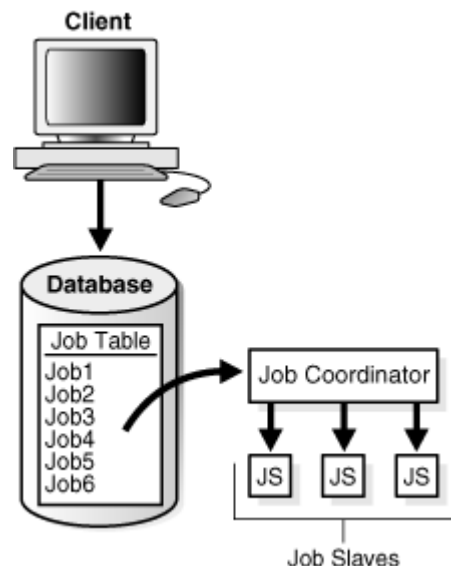


Figure: Scheduler Components

The Job Table

- The **job table** is a container for all the jobs, including those run from pluggable databases, with one table for each database. The job table stores information for all jobs such as the owner name or the level of logging. You can find this information in the *_SCHEDULER_JOBS views.
- Jobs are database objects, and therefore, can accumulate and take up too much space. To avoid this, job objects are automatically dropped by default after completion. This behavior is controlled by the auto_drop job attribute

The Job Coordinator

- The job coordinator, under the control of the database, controls and starts job slaves, making use of the information in the job table.
- The job coordinator background process (cjqNNN) starts automatically and stops on an as-needed basis. At database startup, the job coordinator is not started, but the database does monitor whether there are any jobs to be executed, or windows to be opened in the near future. If so, it starts the coordinator.
- As long as there are jobs or windows running, the coordinator continues to run. After there has been a certain period of Scheduler inactivity and there are no jobs or windows scheduled in the near future, the coordinator is automatically stopped.
- When the database determines whether to start the job coordinator, it takes the service affinity of jobs into account. For example, if there is only one job scheduled in the near future and this job belongs to a job class that has service affinity for only two out of the four Oracle RAC instances, only the job coordinators for those two instances are started.

Job Coordinator Actions

- ☞ The job coordinator performs several actions.
- ☞ The coordinator looks at the root database and all the PDBs and selects jobs based on the job priority, the job scheduled start time, and the availability of resources to run the job. The latter criterion depends on the consumer group of the job and the resource plan currently in effect. The coordinator makes no attempt to be fair to every PDB. The only way to ensure that jobs from a PDB are not starved is to allocate enough resources to it.
- ☞ The job coordinator:
 - Controls and spawns the job slaves
 - Queries the job table
 - Picks up jobs from the job table on a regular basis and places them in a memory cache. This improves performance by reducing trips to the disk
 - Takes jobs from the memory cache and passes them to job slaves for execution
 - Cleans up the job slave pool when slaves are no longer needed
 - Goes to sleep when no jobs are scheduled

- Wakes up when a new job is about to be executed or a job was created using the CREATE_JOB procedure
 - Upon database, startup after an abnormal database shutdown, recovers any jobs that were running.
- ☞ You do not need to set the time that the job coordinator checks the job table; the system chooses the time frame automatically.
- ☞ One job coordinator is used per instance. This is also the case in Oracle RAC environments.

Job slaves

- ☞ Job slaves execute the jobs you submit.
- ☞ They are awakened by the job coordinator when it is time for a job to be executed. They gather metadata to run the job from the job table.
- ☞ When a job is picked for processing, the job slave does the following:
1. Gathers all the metadata needed to run the job, for example, program arguments and privilege information.
 2. Starts a database session as the owner of the job, starts a transaction, and then starts executing the job.
For jobs that are run from a pluggable database (PDB), the slave process switches to the PDB that the job belongs to and then executes it.
 3. Once the job is complete, the slave commits and ends the transaction.
 4. Closes the session.
- ☞ The slaves perform several operations after a job completes. When a job is done, the slaves do the following:
- Reschedule the job if required.
 - Update the state in the job table to reflect whether the job has completed or is scheduled to run again.
 - Insert an entry into the job log table.
 - Update the run count, and if necessary, failure and retry counts.
 - Clean up.
 - Look for new work (if none, they go to sleep).
- ☞ The Scheduler dynamically sizes the slave pool as required

Scheduler Workflow

Scheduler workflow consists of following activities

Defining Programs:

Create a program if you have a repetitive task that you want to use across multiple jobs.

Creating Schedules:

Define a schedule that specifies when the job should run.

Creating Jobs:

Create a job that uses the program and schedule.

Creating Job Classes:

Define job classes to manage and group jobs.

Creating Windows and Window Groups:

Define windows to control when jobs or resource plans are active

Creating Chains:

Define chains for complex job workflows.

Monitoring and Managing Jobs

To monitor and manage Scheduler jobs, you can use views such as:

- ☞ DBA_SCHEDULER_JOBS
- ☞ DBA_SCHEDULER_JOB_LOG
- ☞ DBA_SCHEDULER_RUNNING_JOBS
- ☞ DBA_SCHEDULER_JOB_RUN_DETAILS

Creating a Job

- We create one or more jobs using the DBMS_SCHEDULER.CREATE_JOB or DBMS_SCHEDULER.CREATE_JOBS procedures or Cloud Control.
- We use the CREATE_JOB procedure to create a single job. This procedure is overloaded to enable you to create different types of jobs that are based on different

objects. You can create multiple jobs in a single transaction using the CREATE_JOBS procedure.

- We must have the CREATE JOB privilege to create a job in your own schema, and the CREATE ANY JOB privilege to create a job in any schema except SYS.

Example

- The following example demonstrates creating a database job called *update_sales*, which calls a package procedure in the OPS schema that updates a sales summary table:

BEGIN

```
DBMS_SCHEDULER.CREATE_JOB (
  job_name          => 'update_sales',
  job_type          => 'STORED_PROCEDURE',
  job_action        => 'OPS.SALES_PKG.UPDATE_SALES_SUMMARY',
  start_date        => '28-APR-08 07.00.00 PM Australia/Sydney',
  repeat_interval   => 'FREQ=DAILY;INTERVAL=2', /* every other day */
  end_date          => '20-NOV-08 07.00.00 PM Australia/Sydney',
  auto_drop         => FALSE,
  job_class         => 'batch_update_jobs',
  comments          => 'My new job');
```

END;

/

Assuming successful execution, this code will create a new Scheduler job named 'update_sales' that will run the OPS.SALES_PKG.UPDATE_SALES_SUMMARY stored procedure every other day between the specified start and end dates.

Creating Job Classes

- We create a job class using the CREATE_JOB_CLASS procedure in the DBMS_SCHEDULER package or Cloud Control. Job classes are always created in the SYS schema.
- The following statement creates a job class for all finance jobs:

BEGIN

```
DBMS_SCHEDULER.CREATE_JOB_CLASS (
  job_class_name      => 'finance_jobs',
  resource_consumer_group => 'finance_group');
```

END;

/

All jobs in this job class are assigned to the finance_group resource consumer group.

To query job classes, use the *_SCHEDULER_JOB_CLASSES views.

Creating Schedules

- You create schedules by using the CREATE_SCHEDULE procedure in the DBMS_SCHEDULER package or Cloud Control.
- Schedules are created in the schema of the user creating the schedule, and are enabled when first created. You can create a schedule in another user's schema. Once a schedule has been created, it can be used by other users. The schedule is created with access to PUBLIC. Therefore, there is no need to explicitly grant access to the schedule. The following example create a schedule:

```
BEGIN
```

```
DBMS_SCHEDULER.CREATE_SCHEDULE (
  schedule_name      => 'my_stats_schedule',
  start_date         => SYSTIMESTAMP,
  end_date           => SYSTIMESTAMP + INTERVAL '30' day,
  repeat_interval    => 'FREQ=HOURLY; INTERVAL=4',
  comments           => 'Every 4 hours');
```

```
END;
```

```
/
```

Explanation

1. **schedule_name:** The name of the schedule.
2. **start_date:** When the schedule starts, set to the current timestamp (SYSTIMESTAMP).
3. **end_date:** When the schedule ends, set to 30 days from the current timestamp.
4. **repeat_interval:** Specifies the frequency of the schedule. In this case, the job will repeat every 4 hours (FREQ=HOURLY; INTERVAL=4).
5. **comments:** Descriptive comment for the schedule.

Key Points

- **Start Date:** `SYSTIMESTAMP` returns the current date and time with time zone.
- **End Date:** `SYSTIMESTAMP + INTERVAL '30' DAY` calculates the date and time 30 days from now.
- **Repeat Interval:** `'FREQ=HOURLY; INTERVAL=4'` sets the job to run every 4 hours.

Using Time Based Schedule

- Time-based scheduling in Oracle Scheduler allows you to specify exactly when a job should run, using flexible date and time specifications. Here, we will go through creating and managing time-based schedules in Oracle Scheduler.
- Time-based schedules are the most common way to define execution patterns for jobs in Oracle Scheduler.

Creating a Time-Based Schedule

Let's create a time-based schedule that runs a job at specific times of the day.

Example 1: Daily at Specific Times

Suppose you want a job to run every day at 2:00 AM, 6:00 AM, 10:00 AM, 2:00 PM, 6:00 PM, and 10:00 PM. You can define this schedule as follows:

```
BEGIN

  DBMS_SCHEDULER.CREATE_SCHEDULE (

    schedule_name    => 'daily_times_schedule',

    repeat_interval  => 'FREQ=DAILY; BYHOUR=2,6,10,14,18,22',

    comments         => 'Runs every day at 2AM, 6AM, 10AM, 2PM,
6PM, and 10PM'

  );

END;

/
```

Example 2: Weekly on Specific Days and Times

To create a schedule that runs a job every Monday, Wednesday, and Friday at 9:00 AM and 5:00 PM:

```
BEGIN

  DBMS_SCHEDULER.CREATE_SCHEDULE (

    schedule_name    => 'weekly_schedule',

    repeat_interval => 'FREQ=WEEKLY; BYDAY=MON,WED,FRI; BYHOUR=9,17',

    comments => 'Runs every Monday, Wednesday, and Friday at 9AM and 5PM'

  );

END;

/
```

Key Parameters in Time-Based Scheduling

- **FREQ:** Defines the frequency (e.g., DAILY, WEEKLY, MONTHLY).
 - **BYHOUR:** Specifies the hours when the job should run.
 - **BYDAY:** Specifies the days when the job should run.
 - **BYMINUTE:** Specifies the minutes past the hour when the job should run.
 - **BYMONTH:** Specifies the months during which the job should run.
 - **BYMONTHDAY:** Specifies the days of the month when the job should run.
- Time-based schedules in Oracle Scheduler provide a powerful and flexible way to automate job execution at specific times and intervals.

Another Example of a time based Schedule

For more complex scheduling, you can combine several parameters. For instance, to run a job every Monday and Thursday at 9:15 AM and 4:45 PM:

```
BEGIN

    DBMS_SCHEDULER.DROP_SCHEDULE('complex_schedule');

END;

/

BEGIN

    DBMS_SCHEDULER.CREATE_SCHEDULE (

        schedule_name    => 'complex_schedule',

        repeat_interval  => 'FREQ=WEEKLY; BYDAY=MON,THU; BYHOUR=9,16; BYMINUTE=15,45',

        comments        => 'Updated schedule to run every Monday and Thursday at 9:15AM and 4:45PM'

    );

END;

/
```

Event Based Schedules

- An **event** is a message one application or system process sends to another to indicate that some action or occurrence has been detected. An event is **raised** (sent) by one application or process, and **consumed** (received) by one or more applications or processes.
- In Oracle Scheduler, event-based scheduling allows you to trigger job executions based on the occurrence of specific events within the database or system, rather than relying solely on pre-defined times. This offers more flexibility for automating tasks that depend on dynamic conditions.
- Event-based scheduling in Oracle Scheduler allows jobs to be triggered based on specific events, rather than at predefined times. This is useful for scenarios where tasks need to respond to changes or occurrences within the database or external systems.
- The Scheduler **consumes two kinds of events**:

- *Events that your application raises*

An application can raise an event to be consumed by the Scheduler. The Scheduler reacts to the event by starting a job. For example, when an inventory tracking system notices that the inventory has gone below a certain threshold, it can raise an event that starts an inventory replenishment job.

- *File arrival events that a file watcher raises*

You can create a file watcher, a Scheduler object introduced in Oracle Database 11g Release 2 (11.2), to watch for the arrival of a file on a system. You can then configure a job to start when the file watcher detects the presence of the file. For example, a data warehouse for a chain of stores loads data from end-of-day revenue reports which are uploaded from the stores. The data warehouse load job starts each time a new end-of-day report arrives.

Creating an Event Schedule

- ☑ You can create a schedule that is based on an event. You can then reuse the schedule for multiple jobs. To do so, use the `CREATE_EVENT_SCHEDULE` procedure, or use Cloud Control.
- ☑ The following example creates an event schedule:

```
BEGIN
  DBMS_SCHEDULER.CREATE_EVENT_SCHEDULE (
    schedule_name    => 'inventory_events_schedule',
    start_date       => SYSTIMESTAMP,
    event_condition  => 'tab.user_data.event_type = ''LOW_INVENTORY''',
    queue_spec       => 'inv_events_q, inv_agent1');
END;
/
```

Explanation of Parameters

1. **schedule_name:** The name of the event schedule.
2. **start_date:** The timestamp when the event schedule becomes active.

3. **event_condition:** The condition that must be met for the schedule to trigger. This is typically an expression evaluated against the event data.
4. **queue_spec:** Specifies the queue and agent to be used. This is in the format
`queue_name, agent_name.`

Event-based Job

- Your application can raise an event to notify the Scheduler to start a job. A job started in this way is referred to as an event-based job.
- You can create a named schedule that references an event instead of containing date, time, and recurrence information. If a job is given such a schedule (an event schedule), the job runs when the event is raised.
- To raise an event to notify the Scheduler to start a job, your application enqueues a message onto an Oracle Database Advanced Queuing queue that was specified when setting up the job. When the job starts, it can optionally retrieve the message content of the event.
- To create an event-based job, you must set these two additional attributes:
 — `queue_spec`

A queue specification that includes the name of the queue where your application enqueues messages to raise job start events, or in the case of a secure queue, the queue name followed by a comma and the agent name.

— `event_condition`

A conditional expression based on message properties that must evaluate to TRUE for the message to start the job. The expression must have the syntax of an Oracle Database Advanced Queuing rule. Accordingly, you can include user data properties in the expression, provided that the message payload is an object type, and that you prefix object attributes in the expression with `tab.user_data`.

- You can specify `queue_spec` and `event_condition` as inline job attributes, or you can create an **event schedule** with these attributes and point to this schedule from the job

Passing Event Messages into an Event-Based Job

Through a metadata argument, the Scheduler can pass the message content of the event to the event-based job that started the job.

The following rules apply:

- ☑ The job must use a named program of type `STORED_PROCEDURE`.
- ☑ One of the named program arguments must be a metadata argument with `metadata_attribute` set to `EVENT_MESSAGE`.
- ☑ The stored procedure that implements the program must have an argument at the position corresponding to the metadata argument of the named program. The argument type must be the data type of the queue where your application queues the job-start event.
- ☑ If you use the `RUN_JOB` procedure to manually run a job that has an `EVENT_MESSAGE` metadata argument, the value passed to that argument is `NULL`.
- ☑ The following example shows how to construct an event-based job that can receive the event message content:

-- Step 1: Create the Stored Procedure

```
CREATE OR REPLACE PROCEDURE my_stored_proc (event_msg IN
event_queue_type)
AS
BEGIN
    -- retrieve and process message body
END;
/
```

-- Step 2: Create the Program

```
BEGIN
    DBMS_SCHEDULER.CREATE_PROGRAM (
        program_name => 'my_prog',
        program_action=> 'my_stored_proc',
        program_type => 'STORED_PROCEDURE',
        number_of_arguments => 1,
        enabled => FALSE) ;
```

```

DBMS_SCHEDULER.DEFINE_METADATA_ARGUMENT (
    program_name => 'my_prog',
    argument_position => 1,
    metadata_attribute => 'EVENT_MESSAGE') ;

DBMS_SCHEDULER.ENABLE ('my_prog');
EXCEPTION
    WHEN others THEN RAISE ;
END ;
/

-- Step 3: Create the Event Schedule
BEGIN
    DBMS_SCHEDULER.CREATE_EVENT_SCHEDULE (
        schedule_name    => 'my_evt_sch',
        start_date        => SYSTIMESTAMP,
        event_condition => 'tab.user_data.event_type = ''LOW_INVENTORY''',
        queue_spec        => 'inv_events_q, inv_agent1'
    );
END;
/

-- Step 4: Create the Job
BEGIN
    DBMS_SCHEDULER.CREATE_JOB (
        job_name => 'my_evt_job' ,
        program_name => 'my_prog',
        schedule_name => 'my_evt_sch',
        enabled => true,
        auto_drop => false) ;
EXCEPTION
    WHEN others THEN RAISE ;
END ;
/

```

Explanation

- **Stored Procedure (my_stored_proc):** This procedure processes the event message. You can expand the body to include actual message processing logic.
- **Program (my_prog):** The program encapsulates the action to be taken (the stored procedure). It has one argument for the event message, defined by `DEFINE_METADATA_ARGUMENT`.
- **Event Schedule (my_evt_sch):** This schedule triggers the job based on a specified event condition.
- **Job (my_evt_job):** The job uses the program and the event schedule. It is enabled to run whenever the event condition is met.

Raising an Event

To test the event-based job, you can raise an event manually:

```
BEGIN

  DBMS_SCHEDULER.raise_event(

    event_queue    => 'inv_events_q',

    event_condition => 'LOW_INVENTORY',

    event_data     => 'Low inventory detected in warehouse'

  );

END;

/
```

Another Example Using File watcher

To start a job automatically when a file arrives on a system, Oracle Scheduler provides a feature called File Watcher. The File Watcher monitors a directory for file system events, such as file creation, modification, and deletion, and triggers jobs based on these events. Here's how you can set it up:

Step 1: Create the File Watcher

First, we'll set up the file watcher to monitor a directory for file creation events. `BEGIN`

```
-- Create a credential for the file watcher

DBMS_SCHEDULER.create_credential (

    credential_name => 'file_watcher_cred',

    username        => 'your_os_username',

    password        => 'your_os_password'

);

-- Create the file watcher

DBMS_SCHEDULER.create_file_watcher (

    file_watcher_name => 'file_watcher1',

    directory_path    => '/path/to/directory',

    file_name          => '*.txt', -- Specify the file pattern if needed

    credential_name    => 'file_watcher_cred',

    destination       => NULL, -- You can specify a destination queue here
if needed

    enabled            => TRUE

);

END;

/
```

Step 2: Create the Program and Job

Next, we'll create a program and a job that will be triggered when the file arrives.

```
BEGIN

  -- Create the program to process the file

  DBMS_SCHEDULER.create_program (

    program_name    => 'file_arrival_program',

    program_type    => 'PLSQL_BLOCK',

    program_action  => 'BEGIN process_file(:file_path); END;',

    number_of_arguments => 1,

    enabled         => FALSE

  );

  -- Define the metadata argument for the file path

  DBMS_SCHEDULER.define_metadata_argument (

    program_name    => 'file_arrival_program',

    argument_position => 1,

    metadata_attribute => 'FILE_PATH',

    default_value    => NULL

  );

  -- Enable the program

  DBMS_SCHEDULER.enable ('file_arrival_program');

END;

/

BEGIN

  -- Create the job to execute the program

  DBMS_SCHEDULER.create_job (

    job_name        => 'file_arrival_job',

    program_name    => 'file_arrival_program',
```

```

event_condition => 'file_watcher1.file_event_type = ''FILE_CREATED'',
queue_spec      => 'file_watcher1',
enabled         => TRUE,
auto_drop       => FALSE
);
END;
/

```

Step 3: Create the Procedure to Process the File

Next, create the procedure that will process the file when it arrives.

```

CREATE OR REPLACE PROCEDURE process_file (file_path IN VARCHAR2)
AS
    file_handle UTL_FILE.FILE_TYPE;
    file_contents VARCHAR2(32767);
BEGIN
    -- Open the file
    file_handle := UTL_FILE.FOPEN('YOUR_DIRECTORY', file_path, 'R');

    -- Read file contents
    UTL_FILE.GET_LINE(file_handle, file_contents);

    -- Close the file
    UTL_FILE.FCLOSE(file_handle);

    -- Process file contents
    DBMS_OUTPUT.PUT_LINE('File contents: ' || file_contents);

    -- Add your business logic here

```

```

EXCEPTION

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('Error processing file: ' || SQLERRM);

END;

/

```

Step 4: Testing the Setup

You can test the setup by manually copying a file into the monitored directory. For example:

```
cp /path/to/yourfile.txt /path/to/directory
```

This action should trigger the file watcher, which in turn triggers the job `file_arrival_job` to execute the program `file_arrival_program` to process the file.

Monitoring and Managing Event-Based Jobs

You can monitor the event-based job and its executions by querying the relevant Scheduler views.

```

SELECT job_name, state, event_condition, queue_spec
FROM dba_scheduler_jobs
WHERE job_name = 'PROCESS_LOW_INVENTORY';

```

To see detailed run information:

```

SELECT log_id, job_name, status, actual_start_date, run_duration
FROM dba_scheduler_job_run_details
WHERE job_name = 'PROCESS_LOW_INVENTORY';

```

Follow the [link](https://docs.oracle.com/en/database/oracle/oracle-database/21/admin/scheduling-jobs-with-oracle-scheduler.html#GUID-281B7FC1-6134-4E1F-83B0-082D588F21B6) for detail: <https://docs.oracle.com/en/database/oracle/oracle-database/21/admin/scheduling-jobs-with-oracle-scheduler.html#GUID-281B7FC1-6134-4E1F-83B0-082D588F21B6>